

CMU 10-703: Deep Reinforcement Learning

# Lecture 4: Advantage Actor-Critic and Evolutionary Policy Search

---

Max Simchowitz

Carnegie Mellon University

1.

# Advantage Estimation

---

CMU 10-703: Deep Reinforcement Learning

# Actor-Critic Components

## Critic

Learn the state value  $V_{\phi}^{\pi}(s)$  from policy data.

$$\hat{A}_t = \hat{Q}_t - V_{\phi}^{\pi}(s_t)$$

## Actor

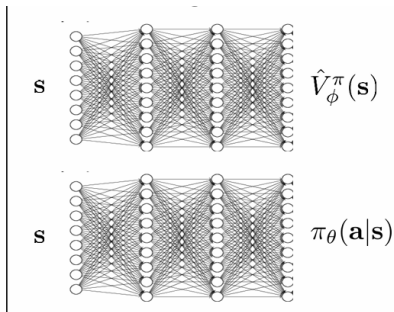
Increase the probability of actions with positive estimated advantage.

$$\hat{g} = \frac{1}{N} \sum_{i,t} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) \hat{A}_t^{(i)}$$

The critic learns a value function; the actor uses that value function to estimate which sampled actions were better than expected.

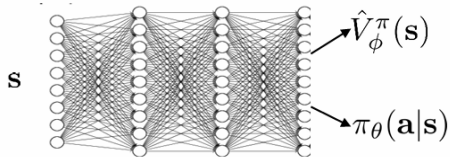
# Actor-Critic Architectures

## Separate networks



Independent actor and critic representations.

## Shared representation



One feature trunk with policy and value heads.

Actor and critic require distinct outputs, but they may share the features used to produce them.

# Multi-Step Advantage Estimates

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{\pi}[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t, a_t]$$

One-step bootstrapping replaces the unknown future return with the critic's next-state prediction.

# Multi-Step Advantage Estimates

$$\begin{aligned} Q^\pi(s_t, a_t) &= \mathbb{E}_\pi[r_t + \gamma V^\pi(s_{t+1}) \mid s_t, a_t] \\ &= \mathbb{E}_\pi \left[ \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V^\pi(s_{t+n}) \mid s_t, a_t \right] \end{aligned}$$

Repeated substitution yields an  $n$ -step target that mixes sampled rewards with a terminal bootstrap.

# Multi-Step Advantage Estimates

$$\begin{aligned} Q^\pi(s_t, a_t) &= \mathbb{E}_\pi[r_t + \gamma V^\pi(s_{t+1}) \mid s_t, a_t] \\ &= \mathbb{E}_\pi \left[ \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V^\pi(s_{t+n}) \mid s_t, a_t \right] \end{aligned}$$

$$\hat{A}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi^\pi(s_{t+n}) - V_\phi^\pi(s_t)$$

Subtracting the current-state value converts the target into an advantage estimate.

# Estimator Bias and Variance

| Estimate    | Target                         | Main benefit                   | Main cost                             |
|-------------|--------------------------------|--------------------------------|---------------------------------------|
| Monte Carlo | Complete reward-to-go          | No bootstrap error             | High variance; waits for termination  |
| $n$ -step   | $n$ rewards, then $V_\phi$     | Adjustable tradeoff            | Depends on critic after $n$ steps     |
| One-step TD | $r_t + \gamma V_\phi(s_{t+1})$ | Low latency and lower variance | More bootstrap and approximation bias |

$n = 1 \longleftrightarrow$  short bootstrap,       $n \rightarrow T - t \longleftrightarrow$  Monte Carlo return

Rollout length controls how much the estimate trusts sampled rewards versus the critic.

# One-Step TD Advantage

**TD advantage.** Use the one-step Bellman residual as a sampled estimate of action advantage.

$$\hat{A}_t^{\text{TD}} = r_t + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t)$$

$$L_V(\phi) = \frac{1}{N} \sum_{i,t} \left[ r_t^{(i)} + \gamma V_{\bar{\phi}}^{\pi}(s_{t+1}^{(i)}) - V_{\phi}^{\pi}(s_t^{(i)}) \right]^2$$

The target parameters  $\bar{\phi}$  are held fixed while differentiating the critic loss.

The critic is trained to predict  $V^{\pi}$ ; its Bellman residual supplies the actor's advantage estimate.

# Advantage Actor-Critic Update

1. Collect transitions or short rollouts with  $\pi_{\theta}(a \mid s)$ .

The current actor determines the data available to both updates.

# Advantage Actor-Critic Update

1. Collect transitions or short rollouts with  $\pi_{\theta}(a \mid s)$ .
2. Fit  $V_{\phi}^{\pi}$  using Monte Carlo,  $n$ -step, or TD targets.

The critic converts sampled experience into a state-dependent reference value.

# Advantage Actor-Critic Update

1. Collect transitions or short rollouts with  $\pi_\theta(a \mid s)$ .
2. Fit  $V_\phi^\pi$  using Monte Carlo,  $n$ -step, or TD targets.
3. Compute  $\hat{A}_t$  from the chosen value target.

$$\hat{U}(\theta) = \frac{1}{N} \sum_{i,t} \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \hat{A}_t^{(i)}$$

The estimated advantage becomes the scalar weight on each policy score.

# Advantage Actor-Critic Update

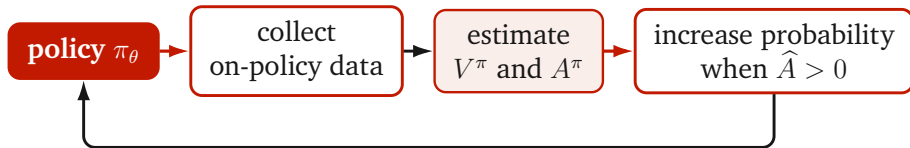
1. Collect transitions or short rollouts with  $\pi_\theta(a \mid s)$ .
2. Fit  $V_\phi^\pi$  using Monte Carlo,  $n$ -step, or TD targets.
3. Compute  $\hat{A}_t$  from the chosen value target.

$$\hat{U}(\theta) = \frac{1}{N} \sum_{i,t} \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \hat{A}_t^{(i)}$$

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \hat{U}(\theta), \quad \phi \leftarrow \phi - \alpha_\phi \nabla_\phi L_V(\phi)$$

Alternating critic descent and actor ascent implements approximate policy improvement.

# Policy Iteration Interpretation



## Evaluation

Fit the critic under the current policy.

## Improvement

Shift probability toward actions that beat the critic's baseline.

## Iteration

Recollect data because the policy and state distribution changed.

Actor-critic interleaves approximate policy evaluation with small, advantage-weighted policy improvements.

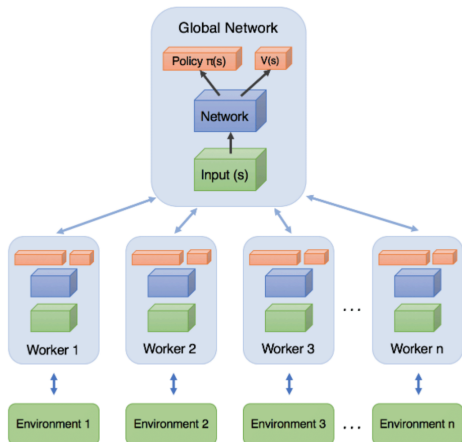
# 2.

## Parallel Actor-Critic

---

CMU 10-703: Deep Reinforcement Learning

# Parallel Experience Collection



## Multiple environment streams

Workers interact with separate environment instances.

## Shared actor-critic parameters

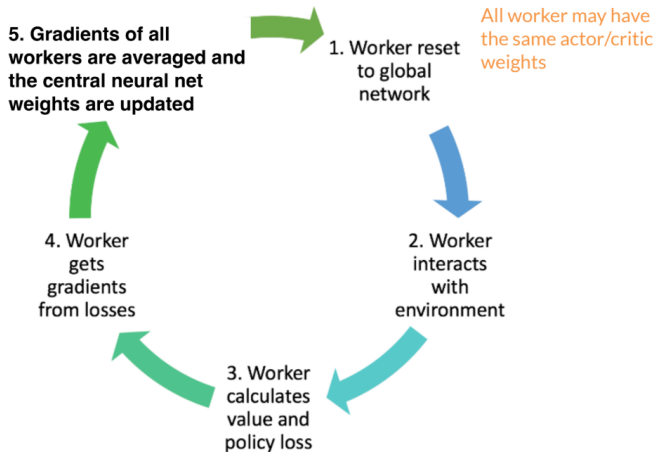
Each stream contributes policy and value updates to a common model.

## Less correlated batches

Concurrent trajectories diversify states, actions, and exploration histories.

Parallel actors improve throughput and reduce the temporal correlation of data presented to the learner.

# Synchronous A2C



## Synchronized workers

All workers begin from the same actor and critic parameters.

## One batched update

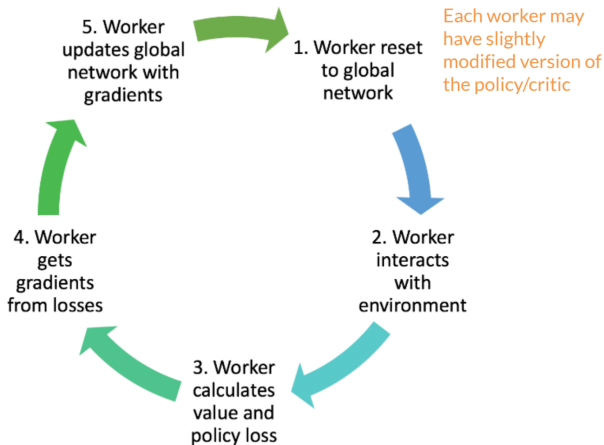
The learner aggregates worker gradients before changing the global model.

## Barrier

Every update waits for the slowest worker in the batch.

A2C turns parallel rollouts into one synchronous actor-critic minibatch.

# Asynchronous A3C



## Local copies

Each worker periodically synchronizes with the global parameters.

## Independent timing

A worker sends its gradient as soon as its rollout segment ends.

## Stale parameters

Local parameters may lag behind the global network.

A3C removes the barrier but permits stale local gradients.

# A2C and A3C Comparison

| Property             | A2C                                         | A3C                                        |
|----------------------|---------------------------------------------|--------------------------------------------|
| Collection           | Parallel rollout segments                   | Parallel rollout segments                  |
| Update timing        | Synchronous gradient aggregation            | Asynchronous worker updates                |
| Worker parameters    | Identical at batch start                    | Potentially stale local copies             |
| Systems tradeoff     | Stable batched execution; straggler barrier | No global barrier; concurrent update noise |
| Typical hardware fit | Vectorized accelerators                     | Independent CPU actor threads              |

A2C and A3C use the same actor-critic estimators; they differ in when worker experience becomes a global update.

# A3C Actor-Learner Thread

1. Copy global parameters into local  $\theta'$  and  $\phi'$ .

Each thread begins a rollout from a snapshot of the shared model.

# A3C Actor-Learner Thread

1. Copy global parameters into local  $\theta'$  and  $\phi'$ .
2. Roll out at most  $t_{\max}$  local steps, or stop at a terminal state.

Short segments bound update latency while retaining multi-step reward information.

# A3C Actor-Learner Thread

1. Copy global parameters into local  $\theta'$  and  $\phi'$ .
2. Roll out at most  $t_{\max}$  local steps, or stop at a terminal state.
3. Bootstrap the segment and sweep backward:

$$R \leftarrow \begin{cases} 0, & \text{terminal state,} \\ V_{\phi'}(s_t), & \text{truncated segment,} \end{cases} \quad R \leftarrow r_i + \gamma R$$

A nonterminal segment bootstraps from its final local value estimate.

# A3C Actor-Learner Thread

1. Copy global parameters into local  $\theta'$  and  $\phi'$ .
2. Roll out at most  $t_{\max}$  local steps, or stop at a terminal state.
3. Bootstrap the segment and sweep backward:

$$R \leftarrow \begin{cases} 0, & \text{terminal state,} \\ V_{\phi'}(s_t), & \text{truncated segment,} \end{cases} \quad R \leftarrow r_i + \gamma R$$

$$g_{\theta} += \nabla_{\theta'} \log \pi_{\theta'}(a_i | s_i) (R - V_{\phi'}(s_i)), \quad g_{\phi} += \nabla_{\phi'} \frac{1}{2} (R - V_{\phi'}(s_i))^2$$

The worker applies actor ascent and critic-loss descent to the shared parameters.

3.

# Evolutionary Policy Search

---

CMU 10-703: Deep Reinforcement Learning

# Parameter-Space Objective

**Fitness.** Evaluate a complete policy parameter vector by the return of trajectories generated by that policy.

$$\max_{\theta} U(\theta), \quad U(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}, s_0 \sim \mu_0} [R(\tau)]$$

## Policy gradients

Differentiate action log probabilities along sampled trajectories.

## Evolutionary search

Perturb complete parameter vectors and compare scalar fitness.

## Shared objective

Both methods seek parameters with high expected trajectory return.

Evolutionary methods change the optimizer, not the expected-return objective.

# Evolutionary Search Cycle

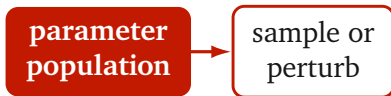
parameter  
population

## Represent

Maintain  
candidates or a  
search distribution.

The optimizer maintains a population or a distribution over policy parameters.

# Evolutionary Search Cycle



## **Represent**

Maintain candidates or a search distribution.

## **Explore**

Generate nearby parameter vectors.

Random perturbations provide directions for parameter-space exploration.

# Evolutionary Search Cycle



## Represent

Maintain candidates or a search distribution.

## Explore

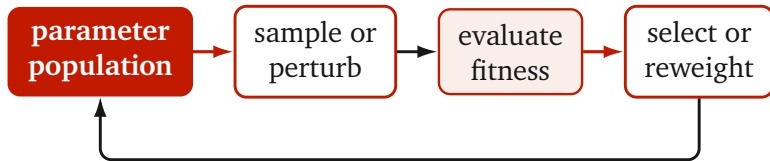
Generate nearby parameter vectors.

## Score

Run complete policy evaluations.

Each candidate receives a scalar fitness from one or more policy rollouts.

# Evolutionary Search Cycle



## **Represent**

Maintain candidates or a search distribution.

## **Explore**

Generate nearby parameter vectors.

## **Score**

Run complete policy evaluations.

## **Update**

Move the search toward better fitness.

Selection or fitness weighting concentrates future samples in better regions.

# Cross-Entropy Method

1. Sample  $n$  candidates from a diagonal Gaussian.

$$\theta_i \sim \mathcal{N}(\mu, \text{diag}(\sigma^2)) , \quad i = 1, \dots, n$$

CEM represents parameter uncertainty with independent coordinate variances.

# Cross-Entropy Method

1. Sample  $n$  candidates from a diagonal Gaussian.
2. Evaluate fitness and retain the top  $m = \lfloor \rho n \rfloor$  candidates.

$$\theta_i \sim \mathcal{N}(\mu, \text{diag}(\sigma^2)), \quad i = 1, \dots, n$$

The elite fraction  $\rho$  determines how aggressively the method selects high-fitness samples.

# Cross-Entropy Method

1. Sample  $n$  candidates from a diagonal Gaussian.
2. Evaluate fitness and retain the top  $m = \lfloor \rho n \rfloor$  candidates.
3. Refit each coordinate of the search distribution to the elites.

$$\theta_i \sim \mathcal{N}(\mu, \text{diag}(\sigma^2)), \quad i = 1, \dots, n$$

$$\mu_j \leftarrow \frac{1}{m} \sum_{i=1}^m \theta'_{i,j}, \quad \sigma_j^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (\theta'_{i,j} - \mu_j)^2 + \eta$$

Refitting the mean and variance shifts future samples toward the elite region while noise  $\eta$  preserves exploration.

# Covariance Matrix Adaptation

**Simplified covariance adaptation.** Use a full-covariance Gaussian so the search distribution can learn correlated parameter directions.

$$\theta_i \sim \mathcal{N}(\mu, C)$$

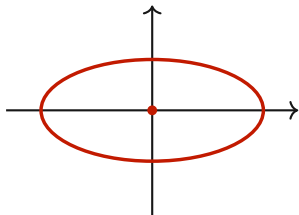
$$\mu \leftarrow \mu + \alpha_\mu \sum_{i=1}^m w_i (\theta'_i - \mu), \quad C \leftarrow (1 - \alpha_C)C + \alpha_C \sum_{i=1}^m w_i (\theta'_i - \mu)(\theta'_i - \mu)^\top$$

Canonical CMA-ES also uses evolution paths and adapts a global step size; this rank- $\mu$  update isolates the covariance idea.

The covariance update learns which parameter combinations vary together among high-fitness candidates.

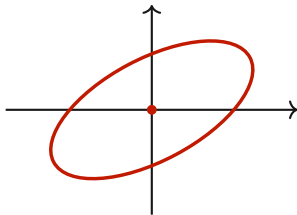
# Diagonal and Full Covariance

CEM: diagonal covariance



Axis-aligned search with  $O(d)$  variance parameters.

CMA-ES: full covariance

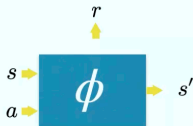


Rotated search with  $O(d^2)$  covariance parameters.

Full covariance can follow coupled directions, but storing and adapting those directions becomes expensive in large policy networks.

# Evolutionary Action-Sequence Search

- A learned model:



- Objective: reaching a desired state:  $\|s_4 - s_*\|$



## Search variable

Candidate action sequences, not persistent policy weights.

## Evaluation

Roll each sequence through a learned dynamics model.

## Execution

Apply the best first action, observe the next state, and replan.

The same zero-order machinery can optimize an action sequence inside model-predictive control instead of optimizing a policy network.

4.

# Natural Evolution Strategies

---

CMU 10-703: Deep Reinforcement Learning

# Search-Distribution Objective

**Search distribution.** Treat the policy parameter vector as a random variable drawn around a learnable mean.

$$\theta \sim P_\mu = \mathcal{N}(\mu, \sigma^2 I_d)$$

$$J(\mu) = \mathbb{E}_{\theta \sim P_\mu}[F(\theta)], \quad \mu^\star = \arg \max_{\mu} J(\mu), \quad F(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

NES smooths the policy-fitness landscape by optimizing expected fitness under a parameter search distribution.

# Score-Function Gradient

$$\nabla_{\mu} J(\mu) = \nabla_{\mu} \int P_{\mu}(\theta) F(\theta) d\theta$$

Begin with expected fitness under the parameter search distribution.

# Score-Function Gradient

$$\begin{aligned}\nabla_{\mu} J(\mu) &= \nabla_{\mu} \int P_{\mu}(\theta) F(\theta) d\theta \\ &= \int \nabla_{\mu} P_{\mu}(\theta) F(\theta) d\theta\end{aligned}$$

Move the mean gradient inside the integral under standard regularity conditions.

# Score-Function Gradient

$$\begin{aligned}\nabla_{\mu} J(\mu) &= \nabla_{\mu} \int P_{\mu}(\theta) F(\theta) d\theta \\ &= \int \nabla_{\mu} P_{\mu}(\theta) F(\theta) d\theta \\ &= \int P_{\mu}(\theta) \nabla_{\mu} \log P_{\mu}(\theta) F(\theta) d\theta\end{aligned}$$

The log-derivative identity converts a probability gradient into a score.

# Score-Function Gradient

$$\begin{aligned}\nabla_{\mu} J(\mu) &= \nabla_{\mu} \int P_{\mu}(\theta) F(\theta) d\theta \\ &= \int \nabla_{\mu} P_{\mu}(\theta) F(\theta) d\theta \\ &= \int P_{\mu}(\theta) \nabla_{\mu} \log P_{\mu}(\theta) F(\theta) d\theta \\ &= \mathbb{E}_{\theta \sim P_{\mu}} [\nabla_{\mu} \log P_{\mu}(\theta) F(\theta)]\end{aligned}$$

The result is an expectation of search-distribution scores weighted by policy fitness.

# Score-Function Gradient

$$\begin{aligned}\nabla_{\mu} J(\mu) &= \nabla_{\mu} \int P_{\mu}(\theta) F(\theta) d\theta \\&= \int \nabla_{\mu} P_{\mu}(\theta) F(\theta) d\theta \\&= \int P_{\mu}(\theta) \nabla_{\mu} \log P_{\mu}(\theta) F(\theta) d\theta \\&= \mathbb{E}_{\theta \sim P_{\mu}} [\nabla_{\mu} \log P_{\mu}(\theta) F(\theta)] \\&\approx \frac{1}{N} \sum_{i=1}^N \nabla_{\mu} \log P_{\mu}(\theta_i) F(\theta_i).\end{aligned}$$

A sampled population provides a Monte Carlo estimator of the search gradient.

# Natural-Gradient Preconditioning

For search-distribution parameters  $\psi$ , define the Fisher information of the distribution score:

$$\mathcal{I}(\psi) = \mathbb{E}_{\theta \sim P_\psi} \left[ \nabla_\psi \log P_\psi(\theta) \nabla_\psi \log P_\psi(\theta)^\top \right].$$

$$\tilde{\nabla}_\psi J = \mathcal{I}(\psi)^{-1} \nabla_\psi J, \quad \psi = \mu, P_\mu = \mathcal{N}(\mu, \sigma^2 I_d) \implies \tilde{\nabla}_\mu J = \sigma^2 \nabla_\mu J.$$

## Full NES

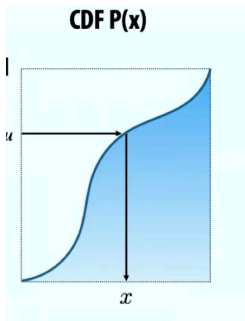
Precondition the search gradient when adapting the distribution's mean, scale, or covariance.

## Fixed isotropic covariance

For the mean, the natural gradient differs only by a scalar absorbed into the learning rate.

The natural gradient measures search-distribution change in probability space rather than raw parameter coordinates.

# Gaussian Sampling



Inverse transform:  $x = F_X^{-1}(u)$  for  $u \sim \mathcal{U}(0, 1)$ .

## Standard-normal noise

$$\epsilon_i \sim \mathcal{N}(0, I_d)$$

## Reparameterized candidate

$$\theta_i = \mu + \sigma \epsilon_i \sim \mathcal{N}(\mu, \sigma^2 I_d)$$

For a full covariance  $\Sigma$ , use  $\theta = \mu + \Sigma^{1/2} \epsilon$ .

Reparameterization separates the shared mean from a reproducible random perturbation direction.

# Gaussian Mean Score

For a fixed isotropic covariance,

$$\log P_{\mu}(\theta) = -\frac{\|\theta - \mu\|^2}{2\sigma^2} + \text{const}$$

$$\nabla_{\mu} \log P_{\mu}(\theta) = \frac{\theta - \mu}{\sigma^2} = \frac{\epsilon}{\sigma}, \quad \theta = \mu + \sigma\epsilon$$

## Direction

The perturbation  $\epsilon$  identifies where the candidate lies relative to the mean.

## Weight

Fitness determines whether the update follows or opposes that perturbation.

For a Gaussian mean, the search-distribution score is the normalized parameter perturbation.

# Monte Carlo NES Estimator

Draw  $\epsilon_i \sim \mathcal{N}(0, I_d)$  and evaluate  $\theta_i = \mu + \sigma \epsilon_i$ .

$$\widehat{\nabla_{\mu} J} = \frac{1}{N} \sum_{i=1}^N F(\theta_i) \frac{\theta_i - \mu}{\sigma^2} = \frac{1}{N\sigma} \sum_{i=1}^N F(\mu + \sigma \epsilon_i) \epsilon_i$$

The update is a fitness-weighted average of random directions in policy-parameter space.

# Monte Carlo NES Estimator

Draw  $\epsilon_i \sim \mathcal{N}(0, I_d)$  and evaluate  $\theta_i = \mu + \sigma \epsilon_i$ .

$$\widehat{\nabla_{\mu} J} = \frac{1}{N} \sum_{i=1}^N F(\theta_i) \frac{\theta_i - \mu}{\sigma^2} = \frac{1}{N\sigma} \sum_{i=1}^N F(\mu + \sigma \epsilon_i) \epsilon_i$$

## Two-sample instance

$$\widehat{\nabla_{\mu} J} = \frac{1}{2\sigma} [F(\theta_1)\epsilon_1 + F(\theta_2)\epsilon_2]$$

Even two candidates illustrate the rule: higher-fitness perturbations contribute more strongly to the mean update.

# NES Optimization Loop

1. Sample perturbations  $\epsilon_i \sim \mathcal{N}(0, I_d)$ .

A population is represented by perturbations around the current search mean.

# NES Optimization Loop

1. Sample perturbations  $\epsilon_i \sim \mathcal{N}(0, I_d)$ .
2. Evaluate each candidate policy  $\theta_i = \mu_t + \sigma \epsilon_i$ .

Each perturbation requires one or more complete policy rollouts to estimate fitness.

# NES Optimization Loop

1. Sample perturbations  $\epsilon_i \sim \mathcal{N}(0, I_d)$ .
2. Evaluate each candidate policy  $\theta_i = \mu_t + \sigma \epsilon_i$ .
3. Aggregate the fitness-weighted perturbations.

$$\hat{g}_t = \frac{1}{N\sigma} \sum_{i=1}^N F(\theta_i) \epsilon_i$$

All sampled candidates contribute to the gradient estimate, rather than only an elite subset.

# NES Optimization Loop

1. Sample perturbations  $\epsilon_i \sim \mathcal{N}(0, I_d)$ .
2. Evaluate each candidate policy  $\theta_i = \mu_t + \sigma \epsilon_i$ .
3. Aggregate the fitness-weighted perturbations.

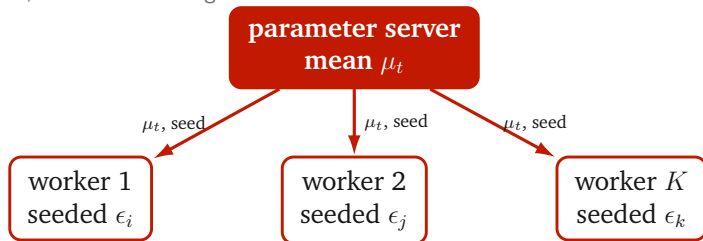
$$\hat{g}_t = \frac{1}{N\sigma} \sum_{i=1}^N F(\theta_i) \epsilon_i$$

$$\mu_{t+1} = \mu_t + \alpha \hat{g}_t$$

The mean update moves the center of the next population toward high-fitness perturbations.

# Distributed NES

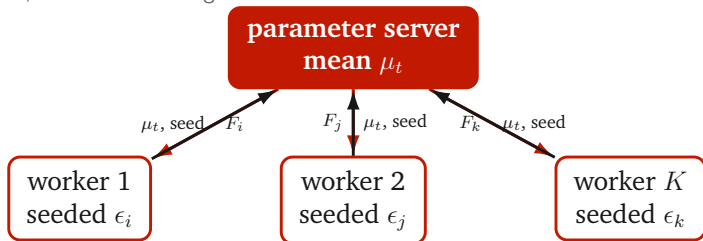
Source: Salimans et al., "Evolution Strategies as a Scalable Alternative to Reinforcement Learning," 2017.



Workers reconstruct assigned perturbations from a shared seed and evaluate candidate policies independently.

# Distributed NES

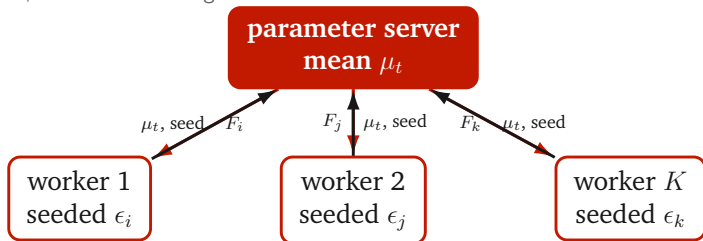
Source: Salimans et al., "Evolution Strategies as a Scalable Alternative to Reinforcement Learning," 2017.



Workers return scalar fitness values rather than transmitting full parameter vectors.

# Distributed NES

Source: Salimans et al., “Evolution Strategies as a Scalable Alternative to Reinforcement Learning,” 2017.



$$\hat{g}_t = \frac{1}{N\sigma} \sum_{i=1}^N F_i \epsilon_i, \quad \mu_{t+1} = \mu_t + \alpha \hat{g}_t$$

The server regenerates perturbations, aggregates the estimator, and broadcasts the updated mean.

5.

# Optimization Comparisons

---

CMU 10-703: Deep Reinforcement Learning

# Policy Gradients and Evolution Strategies

| Property           | Policy-gradient actor-critic                   | Evolution strategy                            |
|--------------------|------------------------------------------------|-----------------------------------------------|
| Random variable    | Actions sampled along a trajectory             | Complete policy parameter vectors             |
| Score              | $\nabla_{\theta} \log \pi_{\theta}(a_t   s_t)$ | $\nabla_{\mu} \log P_{\mu}(\theta)$           |
| Weight             | Return or estimated advantage                  | Scalar policy fitness                         |
| Temporal structure | Uses per-step credit assignment                | Treats each evaluation as a black-box score   |
| Parallelism        | Parallel actors with shared updates            | Independent candidate evaluations             |
| Main pressure      | Noisy trajectory gradients and critic error    | Population evaluation cost in parameter space |

Both estimators use a log-derivative identity; they place the sampling distribution over different objects.

# Lecture Summary

## Advantage estimation

Monte Carlo, multi-step, and TD targets trade target variance against reliance on the critic.

## Parallel actor-critic

A2C aggregates synchronized rollouts; A3C applies updates from independently timed workers.

## Evolutionary search

CEM, CMA-ES, and NES optimize a distribution over policy parameters using scalar fitness.

action-space score  $\longleftrightarrow$  parameter-space score

The central design choice is where to estimate improvement: at each action through an advantage, or across complete policies through fitness.

# Readings

1. Mnih et al., “Asynchronous Methods for Deep Reinforcement Learning,” 2016.
2. Rubinstein and Kroese, *The Cross-Entropy Method*, 2004.
3. Hansen and Ostermeier, “Completely Derandomized Self-Adaptation in Evolution Strategies,” 2001.
4. Wierstra et al., “Natural Evolution Strategies,” 2014.
5. Salimans et al., “Evolution Strategies as a Scalable Alternative to Reinforcement Learning,” 2017.

The readings connect parallel actor-critic implementations with distribution-based black-box optimization.